

浮点数运算

2025/9/17

1. 向偶数舍入（十进制）

原始值（十进制）	向零舍入	向下舍入	向上舍入	向偶数舍入
9.2	9	9	10	9
9.8	9	9	10	10
7.5	7	7	8	8↑
8.5	8	8	9	8↓
-7.5	-7	-8	-7	-8↓

1. 向零舍入 (Towards zero)

- 简单地“砍掉”小数部分，结果向数轴的原点 `0` 靠近。
- 示例: `9.2` 和 `9.8` 都变成 `9`。`-7.5` 舍弃 `.5` 后变成 `-7` (`-7` 比 `-8` 更靠近 `0`)。

2. 向下舍入 (Round down / Floor)

- 舍入到数轴上 **不超过** 原值的最大整数（向负无穷方向取整）。
- 示例: `9.2` 和 `9.8` 都舍入到 `9`。对于负数，`-7.5` 在数轴上比 `-8` 大，所以舍入结果是 `-8`。

3. 向上舍入 (Round up / Ceiling)

- 舍入到数轴上 **不小于** 原值的最小整数（向正无穷方向取整）。
- 示例: 9.2 和 9.8 都舍入到 10。对于负数, -7.5 在数轴上比 -7 小, 所以舍入结果是 -7。

4. 向偶数舍入 (Round to Nearest Even)

- 4舍6入, 对于5 (刚好一半), 向最近的偶数舍入
- 示例:
 - 7.5 : 在 7 和 8 中间, 因为 8 是偶数, 所以舍入到 8 (向上舍入)。
 - 8.5 : 在 8 和 9 中间, 因为 8 是偶数, 所以舍入到 8 (向下舍入)。
 - -7.5 : 在 -7 和 -8 中间, 因为 -8 是偶数, 所以舍入到 -8 (向下舍入)。

2. 向偶数舍入 (二进制)

原始值	舍入划分	舍入行为	舍入行为	舍入结果
10.110011_2	10.110	011_2 小于一半 (因为 $011_2 < 100_2$)	向下舍入 (截断)	10.110_2
11.001101_2	11.001	101_2 大于一半 (因为 $101_2 > 100_2$)	向上舍入 (进位)	100.000_2
10.011100_2	10.011	100_2 恰好一半, 且 LSB 为 1 (奇)	向上舍入 (变为偶)	10.100_2
11.110100_2	11.110	100_2 恰好一半, 且 LSB 为 0 (偶)	向下舍入 (保持偶)	11.110_2

- 二进制下的偶数表示: 最低有效位 (Least Significant Bit) 为0
- 二进制下的中间值: $.1000\dots000$ 划分值LSB后一位为1, 后续都为0, 大于就进位, 小于就舍入, 等于就向最近的偶数舍入 (必在一个奇数和一个偶数的中间)
- 中间值向偶数舍入操作: 最终结果为偶数 $\Leftrightarrow$ 结果LSB为0
 - $LSB=1$: 奇数 $<$ 原始值 $<$ 偶数, 向上舍入 (进位), 得到LSB为0
 - $LSB=0$: 偶数 $<$ 原始值 $<$ 奇数, 向下舍入 (截断), 得到LSB为0

3. 浮点数乘法

使用8位“迷你浮点数”进行演示：1位sign+3位exponent+4位fraction

要计算 -3.625×4.5

$$\text{Bias} = 2^2 - 1 = 3$$

$$-3.625 = (-1)^1 \times 1.101_2 \times 2^1 = \text{B2F}(11001010)$$

$$4.5 = (-1)^0 \times 1.001_2 \times 2^2 = \text{B2F}(01010010)$$

$$\begin{aligned} -3.625 \times 4.5 &= (-1)^{1+0} \times (1.101_2 \times 1.001_2) \times 2^{1+2} \\ &= (-1)^1 \times 1.110101_2 \times 2^3 \end{aligned}$$

舍入后得到 $(-1)^1 \times 1.1101_2 \times 2^3 = \text{B2F}(11101101)$

1. 计算准确结果 $(-1)^s \times M \times 2^E$

1. 取符号位sign $s_1, s_2, s = s_1 \text{ xor } s_2$

2. 加上前导1得到尾数fraction M_1, M_2 :

$$M = M_1 \times M_2$$

3. 计算得到指数exponent e_1, e_2 , 减去Bias得到实际指数 E_1, E_2 , 相加得到结果指数 E :

$$E_1 = e_1 - \text{Bias}, E_2 = e_2 - \text{Bias}, E = E_1 + E_2$$

2. 调整结果

1. $M \geq 2$, 右移 M , $E = E + 1$

2. 对 M 进行舍入 (有进位情况重复步骤1)

3. E 如果在合法范围外, overflow

4. 浮点数加法

使用8位“迷你浮点数”进行演示：1位sign + 3位exponent + 4位fraction $\rightarrow$ Bias = 3。

计算 $6.5 + 0.875$ 。

$$6.5 = (-1)^0 \times 1.101_2 \times 2^2 = B2F(01011010)$$

$$0.875 = (-1)^0 \times 1.11_2 \times 2^{-1} = B2F(00101100)$$

1. 对齐阶码

首先将较小的指数对齐到较大的指数。 $E_{\max} = 2$ ，将 0.875 的尾数右移 $2 - (-1) = 3$ 位。

$$(1.101_2 \times 2^2) + (1.11_2 \times 2^{-1}) = (1.101_2 \times 2^2) + (0.00111_2 \times 2^2)$$

2. 尾数相加（如果是负数则相减）

$$(1.101_2 + 0.00111_2) \times 2^2 = 1.11011_2 \times 2^2$$

3. 调整结果（舍入、规格化）

计算结果 $1.11011_2 \times 2^2$ 已经是规格化形式，但分数位 .11011 超出4位精度，需要舍入。

舍入后得到

$$(-1)^0 \times 1.1110_2 \times 2^2 = B2F(01011110)$$

1. 计算准确结果 $(-1)^s \times M \times 2^E$

1. 阶码 (Exponent) **E**: 取两个数中较大的阶码作为结果的阶码, 即 $E = \max(E1, E2)$ 。
2. 尾数 (Significand) **M**: 将阶码较小的数的尾数右移 $|E1 - E2|$ 位, 然后将两个对齐后的尾数进行带符号加法。
3. 符号 (Sign) **s**: 由尾数加法的结果自然得出。

2. 调整结果

1. 若 $M \geq 2$, 右移M, E增加1 (结果大于等于2, 需右归)。
2. 若 $M < 1$, 左移M, E减少k (结果小于1, 需左归)。
3. 对 M 进行舍入以满足精度要求 (若舍入导致进位, 可能需要重复步骤1)。
4. 检查 E 是否在合法范围外, 判断是否溢出 (overflow)。

5. 浮点数加法特性

1. 加法运算下封闭 (`Inf` 和 `NaN` 仍属于浮点数)
2. 交换律
3. 不满足结合律
 - 原因: 不同顺序相加可能导致先后舍入的精度不同, 从而舍入的值不同
 - 例子: $(3.14 + 1e10) - 1e10 = 0$, $3.14 + (1e10 - 1e10) = 3.14$
4. 加法运算下的单位元(additive identity): 0
5. 加法逆元: 除 `Inf` 和 `NaN` 之外都有加法逆元

6. 单调性 $a \geq b \Rightarrow a + c \geq b + c$ 不一定满足

- 不满足单调性的情况
 - overflow
 1. 计算中不等号两边都溢出至 $+\infty$ ，结果无法比较
 2. 计算中有 `INF` 导致结果为 `NaN`，无法比较

6.浮点数乘法特性

1. 乘法运算下封闭
2. 交换律
3. 不满足结合律
 - 中间产生overflow
 - 舍入精度不一
4. 乘法单位元

5. 不满足乘法分配律

- overflow
- 舍入精度改变

6. 单调性 $a \geq b \wedge c \geq 0 \Rightarrow a * c \geq b * c$ 不一定满足

- overflow

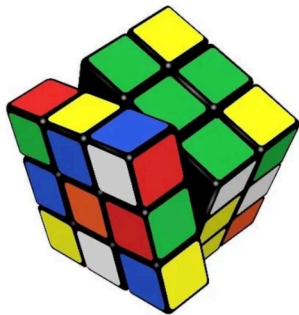
7. 阿贝尔群 (Abelian Group)

1. 群

**When your mom calls
you by your full name**

$$(\mathbb{Z}_3^7 \times \mathbb{Z}_2^{11}) \rtimes ((A_8 \times A_{12}) \rtimes \mathbb{Z}_2)$$

Shit!



群 (group) 是配备了二元运算 (二元运算即满足封闭性) 的集合, 其二元运算需要具有结合律、单位元、逆元素。

1. 正式定义

给定集合 G , 且它配备的二元运算 $\circ : G \times G \rightarrow G$ 满足 (其中运算结果 $\circ(a, b)$ 被简记为 $a \circ b$): [19]

结合律	对所有 $g_1, g_2, g_3 \in G$ 有 $g_1 \circ (g_2 \circ g_3) = (g_1 \circ g_2) \circ g_3$	
左单位元与左逆元素	存在 $e \in G$, 对所有 $g \in G$ 有	$e \circ g = g$
		且存在 $\gamma \in G$ 使得 $\gamma \circ g = e$

的话, 称 $(G, \circ)$ 是一个群。当其配备的二元运算 $\circ$ 不是那么重要时, $(G, \circ)$ 也常常简记为 G 。单位元常称作幺元。

群运算的次序很重要, 也就是说, 等式 $a \circ b = b \circ a$ (交换律) 不一定成立。满足交换律的群称为交换群 (或阿贝尔群, 以尼尔斯·阿贝尔命名), 不满足交换律的群称为非交换群 (非阿贝尔群)。如以下面举例一节的二面体群就不是交换群。

2. 基本性质

1. 单位元唯一

引入以下证明：

若群 $(G, \circ)$ 同时有两个单位元 $e \in G$ 和 $e' \in G$ ，那根据定义里对单位元的定义，对于任意 $g \in G$ 有：

$$g = e \circ g = g \circ e$$

$$g = e' \circ g = g \circ e'$$

这样的话，把 g 分别替换为 e' 和 e 就有

$$e' = e \circ e' = e' \circ e$$

$$e = e' \circ e = e \circ e'$$

所以

$$e' = e$$

2. 逆元唯一

在增添以上函数符号 $e_{(G, \circ)}$ 和新公理的情况下，就可以证明元素的唯一性。

若群 $(G, \circ)$ 的某元素 $g \in G$ 有两个逆元 $\gamma \in G$ 和 $\bar{\gamma} \in G$ ，那根据定义和 $e_{(G, \circ)}$ 的新公理有

$$e_G = g \circ \gamma = \gamma \circ g$$

$$e_G = g \circ \bar{\gamma} = \bar{\gamma} \circ g$$

那这样的话，依据定义里的**结合律**和 $e_{(G, \circ)}$ 的新公理有

$$\begin{aligned} \bar{\gamma} &= \bar{\gamma} \circ e_G \\ &= \bar{\gamma} \circ (g \circ \gamma) \\ &= (\bar{\gamma} \circ g) \circ \gamma \\ &= e_G \circ \gamma \\ &= \gamma \end{aligned}$$

所以任意 $g \in G$ 只拥有一个逆元。这样根据函数符号与唯一性的关系，可以添加一个三元函数符号 $g_{(G, \circ)}^{-1}$ 与以下新公理 (以下的公式式 D 是 “ $(G, \circ)$ 为一群，且 $g \in G'$ 的正式逻辑表述)

$$[\neg D \wedge (g_{(G, \circ)}^{-1} = \emptyset)] \vee [D \wedge (g_{(G, \circ)}^{-1} \circ g = g \circ g_{(G, \circ)}^{-1} = e_G)]$$

这条公理直观上表示，只要“ $(G, \circ)$ 为一群，且 $g \in G'$ ”，就可以用 $g_{(G, \circ)}^{-1}$ 简记“ g 在 $(G, \circ)$ 上对应的那个唯一逆元素”，否则取 $g_{(G, \circ)}^{-1}$ 为空集。

简便起见， $g_{(G, \circ)}^{-1}$ 通常记为 g^{-1} 。

3. 群的幂

若 $(G, \circ)$ 为一群，可以仿造整数指数，对任意 $g \in G$ 作如下关于符号简写的递归定义

$$g^0 := e \quad (1)$$

$$g^1 := g \quad (2)$$

$$\forall n \in \mathbf{Z}, g^{n+1} := g^n \circ g \quad (3)$$

$$\forall n \in \mathbf{Z}, g^{n-1} := g^n \circ g^{-1} \quad (4)$$

4. 减法与除法

有时，某个群的运算可以跟直观上的加法联想在一起，这个运算也可以改记为" $+$ "或" $\oplus$ "，这时也会把 $f + g^{-1}$ 改记为 $f - g$ 或 $f \ominus g$ ，这时会昵称为**减法**。更有甚者， g^{-1} 会被记为 $-g$ 。

类似的，如果群的运算可以跟直观上的乘法联想在一起而改记为" $\times$ "或" $\otimes$ "，这时会把 改记为 或 fg ，并昵称为**除法**；更有甚者， g^{-1} 会被记为 $\frac{1}{g}$ 。

注意以上都是为了**直观理解方便**所规定的简写，并不是断定群的运算必然跟一般实数的加减乘除一模一样。

3. 陪集

在很多情况下，需要认为两个元是等同的，如果它们只相差一个给定子群中的元素。子群 H 定义了左陪集和右陪集，它们可以认为是把 H 平移了一个任意群元素 g 。用符号表示， H 的包含 g 的左和右陪集分别构成集合

$$G/H = \{xH : x \in G\} \text{ 与 } H \backslash G = \{Hx : x \in G\},$$

其中 左陪集 $gH = \{gh : h \in H\}$, 右陪集 $Hg = \{hg : h \in H\}$ 。

同时也可以定义双陪集为 $H \backslash G / K = \{HxK : x \in G, HxK = \{h x k : h \in H, k \in K\}\}$

4. 商群

正规子群 一个子群 H 是群 G 的正规子群, 如果对于 G 中任意元素 g , 都有 $gH = Hg$ 。

设 G 是一个群, $S, T \subseteq G$, 则它们的**乘积** 定义为

$$ST = \{st \mid s \in S, t \in T\}$$

即 ST 是所有 $s \in S$ 与 $t \in T$ 的乘积所组成的集合。

设 N 是群 G 的正规子群, 则 G 关于 N 的商群 (quotient group) 或因子群 (factor group) 是由 G 的所有左陪集组成的集合, 记为 $G/N = \{gN : g \in G\}$ 。

商群的运算定义为陪集之间的“乘法”。

这个运算是闭合的, 因为 $(aN)(bN)$ 是一个左陪集:

$$(aN)(bN) = a(Nb)N = a(bN)N = (ab)NN$$

上述等式用到了 N 的正规性。因为 N 是正规子群, N 在 G 中的左陪集和右陪集相等

2. 阿贝尔群

1. 定义

阿贝尔群 (Abelian group), 又称**交换群**, 是指其二元运算满足**交换律**的群。

一个群 $(A, \circ)$, 若对任意 $a, b \in A$, 都有

$$a \circ b = b \circ a$$

则称 $(A, \circ)$ 为阿贝尔群 (交换群); 否则称为非阿贝尔群 (非交换群)。

2. 基本性质

1. 交换律

2. 阿贝尔群的所有子群都是正规子群

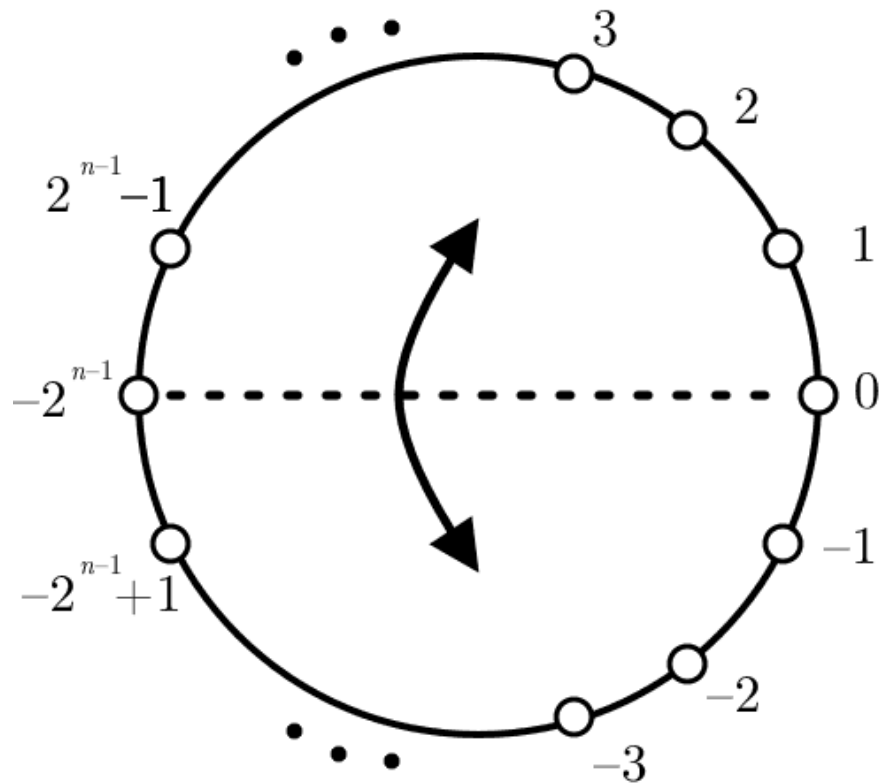
3. 阿贝尔群与二进制补码

为什么二进制补码能和无符号整数保持相同的运算？

- n 位无符号整数 $\mathbf{Z}/2^n\mathbf{Z}$ 的集合 在模 2^n 加法下形成一个群。 n 位带符号整数的集合也形成一个群，运算仍然是“模 2^n 加法”，记为 $\oplus$
- 商群 $\mathbf{Z}/2^n\mathbf{Z}$ 的元素不是整数，而是整数的等价类。我们用 $[x] \in \mathbf{Z}/2^n\mathbf{Z}$ 表示 $x \in \mathbf{Z}$ 的等价类。
 - 例如，“元素1”是等价类 $\{\dots, -2^n + 1, 1, 2^n + 1, \dots\}$
 - 定义等价类的全部意义在于，你可以从等价类中选择一个代表元素，使用该元素进行算术运算，并且得到的结果（等价类）与使用“通常”代表元素时得到的结果相同。换句话说，

$$[x] = [x'] \wedge [y] = [y'] \Rightarrow [x \oplus y] = [x][y] = [x'][y'] = [x' \oplus y']$$
- 因此，任何用于无符号整数的加法运算的电路表示也可以用于所有可能的等价类代表的加法。因为加法电路本质上提供了“模 2^n 加法”运算的表示

- 这也清楚地表明选择 -2^{n-1} 作为最小负数是任意的，可以同样选择 2^{n-1} 作为最大正数，留下 $-2^{n-1} + 1$ 作为最小负数。
 - 当前选择的优点：可以通过符号位判断负数、下溢出...
- 现在我们可以研究n位有符号整数群中的取负 (negation) 运算。
 - 对于除 -2^{n-1} 以外的所有元素 x ， $-x$ 也是一个n位有符号整数
 - 考虑商群中 根据加法逆元的定义， $[-2^{n-1}] \oplus [-2^{n-1}] = [-2^{n-1} \oplus -2^{n-1}] = [0]$ ，所以 $[-2^{n-1}] = -[-2^{n-1}]$
 - 所以在n位有符号整数群中， -2^{n-1} 是自己的逆元，0也是自己的逆元



- 如果将无符号整数的群视为一个圆，则取反操作可以看作是穿过0和圆心的线的反射

8. 判断不等式是否成立

```
int x = ...;  
float f = ...;  
double d = ...;
```

假设 `d` 和 `f` 都不是NaN

表达式	是否成立	原因
<code>x == (int)(float) x</code>	否	一个反例: <code>0x7fffffff</code> 强转到float会因为frac位数不够而向偶数舍入
<code>x == (int)(double) x</code>	是	double的尾数frac有50位>32, int到double可以建立单射, (double)x强转为(int)时不会发生舍入
<code>f == (float)(double) f</code>	是	double的尾数50位>float尾数23位, 可以建立float中非NaN和Inf元素到double中非NaN和Inf元素的单射,

表达式	是否成立	原因
<code>d == (double)(float)d</code>	否	反例：非Inf的double强转为float可能会产生Inf
<code>f == -(-f)</code>	是	操作改变了两次符号位，其余不变，得到的结果和原f一致
<code>2/3 == 2/3.0</code>	否	2/3 是int类型，结果为0，2/3.0结果为float类型，结果为0.666667，比较时会比较(float) 0和0.666667
<code>d < 0.0 ⇒ ((d*2) < 0.0)</code>	是	1. 结果仍为负数 2. 结果为 $-\infty$ ，仍然小于0
<code>d > f ⇒ -f > -d</code>	是	(f-d)与-(f-d)符号相反，其余位不变
<code>d * d >= 0.0</code>	是	两个乘数符号位相同，得到结果符号位 $s = s_1 \text{ xor } s_2 = 0$ ，对INF依然成立

9.浮点数的表示范围

1. float

1+8+23

因为浮点数是关于0对称的，我们先考虑符号位为0的情况

意义	Sign	Exponent	Fraction	十进制值
0	0	00000000	00000000 00000000 00000000	0
最小的非规格化数	0	00000000	00000000 00000000 00000001	1×2^{-149}
最大的非规格化数	0	00000000	11111111 11111111 11111111	$(1 - 2^{-23}) \times 2^{-126}$
最小的规格化数	0	00000001	00000000 00000000 00000000	1×2^{-126}
最大的规格化数	0	11111110	11111111 11111111 11111111	$(2 - 2^{-23}) \times 2^{127}$

2. double

1+11+52

因为浮点数是关于0对称的，我们先考虑符号位为0的情况

意义	Sign	Exponent (11位)	Fraction (52位)	十进制值
0	0	00000000000	全为 0	0
最小的非规格化数	0	00000000000	0...001	1×2^{-1074}
最大的非规格化数	0	00000000000	全为 1	$(1 - 2^{-52}) \times 2^{-1022}$
最小的规格化数	0	00000000001	全为 0	1×2^{-1022}
最大的规格化数	0	11111111110	全为 1	$(2 - 2^{-52}) \times 2^{1023}$

所以 double 的表示范围在 $[-(2 - 2^{-52}) \times 2^{1023}, (2 - 2^{-52}) \times 2^{1023}]$